Automating Safety Critical Software Engineering

Agentic Optimisation and Transformation

Name: Oliver Haythornthwaite

Student ID: s479832

Department: Applied Artificial Intelligence

Date: 15/08/2026

Information categorisation:
Open

Contents

Information categorisation:
Open

Background and Research Aims

Safety critical software defects or failures constitute a risk to the loss of a platform or human life. The assurance of any software system must demonstrate the development process has systematically prevented, detected and resolved defects or failure modes.

In civil airborne safety critical software development, adherence to RTCA DO-178C [1] is substantiated at the project level, through the application of standards derived processes. The production of planning documents, defining such processes, outline how the objectives defined in RTCA DO-178C [1] against lifecycle artefacts, and supplementary standards (330, 331) [2, 3] are met. The aforementioned objectives are scaled in effort, by the Design Assurance Level (DAL), where the level of evidence is required where software failure may lead to a catastrophic condition. Defence Standard 00-055 Part 1 [4] defines RTCA DO-178C [1] as an acceptable means of demonstrating compliance for the mitigation of such conditions. Further the FAA Advisory Circular 20-115D [5] substantiates RTCA DO-178C [1] suitability as a means of compliance, reiterating the responsibility of the integrator, in the satisfaction of applicable objectives.

The execution of these processes to produce such artefacts is highly parallelisable, and automatable dependent on the presence of well-defined requirements at the system and dependency domain level. However, the scope of this issue extends beyond engineering process automation, a certifiable project requires the production of each element contained within Table 1, at minimum (DAL dependent).

DO-178C Reference	Lifecycle Component	Lifecycle Process	DAL
11.1	Plan for Software Aspects of Certification (PSAC)	Planning	A-D
11.2	Software Development Plan (SDP)	Planning	A-D
11.3	Software Verification Plan (SVP)	Planning / verification	A-D
11.4	Software Configuration Management Plan (SCMP)	Planning / Configuration	A-D
11.5	Software Quality Assurance Plan (SQAP)	Planning / QA	A-D
11.6	Software Requirements Standards	Development standards	A-C
11.7	Software Design Standards	Development standards	A-C
11.8	Software Implementation Standards	Development standards	A-C
11.9	Software Requirements Data	Requirements Development	A-D
11.10	Design Description	Software design	A-C
11.11	Source Code	Coding	A-C
11.12	Executable Object Code	Integration	A-D
11.13	Software Verification Cases and Procedures	Verification	A-D
11.14	Software Verification Results	Verification	A-D
11.15	Software Life Cycle Environment Configuration Index	Configuration	A-D
11.16	Software Configuration Index	Configuration	A-D
11.17	Problem Reports	Configuration	A-D
11.18	Software Configuration Management Record	Configuration	A-D
11.19	Software Quality Assurance Records	QA	A-D
11.20	Software Accomplishment Summary (SAS)	Certification liaison	A-D
11.21	Bi-Directional Traceability Evidence	Development / verification	A-D
11.22	Parameter Data Item File	Development / verification	(Where used)

Table 1 - RTCA DO-178C Lifecycle Components

Environmental and Project Context
Bi-directional traceability, requires a cohesive requirement decomposition, supported by through life trace management support, offered in toolsets such as SCADE. This lifecycle, when employed using an MBSE approach, is already optimised from an assurance perspective, as bounded credit for DO-331 can be claimed, via the vendor supplied KCG qualified code generation capability. This credit does not remove the burden of proof around requirement quality, model implementation correctness, or the satisfaction of objective based processes [3, 6, 7].

Manual process automation is therefore supported by the characteristics of the supportive standard, the work is structured, requirements driven and evidence rich in nature, relying upon deterministic toolsets to maintain a standardised approach to the delivery of artefacts. Large Language Models (LLMs) can ingest natural language, code and prompted context, while Retrieval Augmented Generation (RAG) can instead route responses through the projects corpus of information, rather than relying upon model parameters alone [8]. An extension of these approaches is agentic orchestration, where model instantiations can plan, use toolsets and coordinate highly specialised/constrained agents for the fulfilment of a task [25]. Agentic, parallelised planning, development and review of lifecycle activities, with or without human supervision has been achieved.

Considering the proprietary nature of MBSE tools, and so as to smoothly and easily integrate such tools into an agentic workflow, they shall be used to produce associated artefacts, but use must remain compliant with license restrictions and fair use stipulations. A process which considers impacts to established safety case driven evidence substantiation, must be prioritised, to not increase certification effort. The associated airworthiness authority must validate that the defined Acceptable Means of Compliance is substantiated by impact appropriate processes.

Risk and Security
Agentic approaches to software development are rapidly evolving. With the democratisation of Artificial Intelligence (AI) bringing more memory/cache efficient frontier models into the scope of local or on-prem execution, the propositional value of developing a proprietary and certifiable Agentic Model Based Software Engineering (AMBSE) pipeline increases.
However the desirable benefits offered by the technology, introduce technological and commercial risk. The underlying principles of the transformer architecture, means LLM output is probabilistic, sensitive to prompt correctness, and that they can fluently and plausibly answer a question incorrectly. Generated artefacts may therefore appear correct, but hide hallucination induced traceability, functionality or logical errors within a plausibly structured and presented artefact.

Further, post deployment learning and even conversation context, can upset the determinism of answers and reduce repeatability. Should proper consideration not be given to hosting or security concerns, Intellectual Property (IP) or classified information could be exposed through logging, callbacks, adversarial prompting or poisoned retrieval methods/content [21, 22, 27].

On premises deployment allows for tighter control and mitigation of data disclosure/residency problems, however they introduce up front cost, maintenance and security commitments [29]. Access to frontier models provides additional capability where tool integration, reasoning and parameter size is concerned, however such usage potentially risks unauthorised disclosure of export controlled, or otherwise classified information. It is recognised that any schedule level time savings can only be realised, once model hosting, tool integration, model baselining and evaluation has been performed.

Regulatory and Organisational Considerations

The regulations applicable to the utilisation of AI as a development aid, and around the airborne deployment of AI distinctly identify associated challenges. It is not the aim of this report to suggest a means for the assurance or expedition of deployed AI, instead this report is primarily concerned with AI enabled development. Guidance within UK MOD JSPs and the EASA Artificial Intelligence Concept Paper (Proposed Issue 3) [24, 9], whilst concerned with deployed AI, relies upon concepts such as operational domains/design domains, human factors, governance and safety cases, which is also applicable to the enablement piece. EASA and MOD guidance provides useful governance and as such, their recommendations shall be adhered to where applicable, but DO-178C, DO-330 and DO-331 remain as the basis for assurance through Defence Standard 00-055 Part 1 [1-5].

Where no certification credit is claimed for the development aid and where output is fully reviewed via an independently controlled review process, tool qualification may not be required [1, 2, 5]. The cost and engineering ability required to perform such reviewing, is already considered at the project planning stage, according to any assumed DAL rating [1]. The practicable route towards attaining this requires complete integration with proven and qualified toolsets. LLM produced model code, can be imported into SCADE, from which qualified code is generated, using SCADE's KCG facility notwithstanding, the correctness of any generated model must still be verified against requirements [3, 6].

The structure of engineering teams is formed according to the needs of a specific project, its associated budget and timeframe. It is the intent of this review to maintain such a structure, to retain attribution, for the establishment of assurance. It is recognised human review is most useful when involving Suitably Qualified and Experienced Personnel (SQEP), however such reviewers require sufficient context, time and capability to detect an obfuscated defect.

A bias towards automation can cause a tendency to omit information or contrary evidence, primarily whilst under pressure, workload or when using poorly defined performance metrics [10]. Therefore, any safe process transformation, must provably preserve the quality, independence and traceability afforded by any previous iteration. The feasibility of agentic engineering, will be evaluated through analysis of potential improvements which can be gained through the allocation of any number of agentic engineers, prompted, guided and validated in a custom framework by SQEP.

Accordingly, this review establishes if a human led, LLM enabled lifecycle, or a gated Agentic lifecycle can reduce spent time and engineering effort within an MBSE, DO-178C/DO-331 compliant development schedule. The achievement of this effort must be balanced by an evaluation of compliance, the completeness of objective related evidence, the absence of defects, traceability and independence. The intended contribution, is the design of process acceleration and the establishment of an empirical basis to allow the safe transformation of established processes.

State of the Art

Assurance Led Manual Safety Critical Software Development

The context of operation, monetary value and implications of a systems failure, inform the methodologies used to assess the suitability and correctness, of software's design, implementation and verification. Historically, software written in assembly was subjected to additional review around memory usage, efficient instruction and flight testing as means of substantiating means of compliance. With the observed increase in software complexity, provably permutative approaches to verification, and formal methods of software implementation become infeasible. DO-178A defined processes, requiring the correct documentation, review and testing of software, across a structured lifecycle. Following iterations, D0178B and D0178C, shifted to promote compliance to objectives (with objective compliance scaled according to Design Assurance Levels (DAL), through the collection and demonstration of evidence, defined in targeted plans and standards documentation [1, 5].

The traditional software lifecycle, when exacted in an Agile manner, is driven by the delivery of artefacts, which provide the necessary evidence, to claim compliance, against pertinent plans and standards documentation, implementing the means of compliance for high level standards such as DO-178C.

At each stage of delivery, these artefacts are reviewed to confirm compliance against plans and standards, and later, to confirm their functional correctness as per the stated requirements. This is a sequential process, where system level requirements (and associated de-risking activities) are produced, as pre-requisite deliverables, to the commencement of high level requirement (HLR) implementation, just as HLRs, precede the commencement of low level requirement (LLR) implementation.

Processes which increase the quality of any delivered artefact, tend to cost the project more time and money, with effort scaling in orders of magnitude, according to an increase in the Design Assurance Level (DAL) associated with the project / domain.

Resultingly, high integrity software would see considerable cost savings from complete automation, consequently incurring the highest potential penalty as a consequence of failure, in the loss of human life. For this reason, the integration of AI/ML into the domain of safety critical software engineering, is rife with concern and scepticism, no established means of compliance, for deployed AI/ML has been agreed by the military certification authorities. Ongoing work maintains a position of caution, advising against Level 3 (fully autonomous/human out of the loop) operation [9, 23].

LLM Supported Safety Critical Software Development

Current software engineering literature, anecdotal evidence and the prevailing public opinion align in the identification of LLMs as a transformative offer of capability. RAG integration indeed offers a practical route for the grounding of models, in the corpus of project data, bounding responses in that which can be observed as fact [8]. For any LLM exploitation in a safety critical context, an increased level of logging should be performed. Traceability and explicit location references for any decision making shall be treated as a configurable artefact for the purpose of future audit.

Increases in productivity have been measured across the literature, but the magnitude of results is dependent on the setting and context. As part of a bounded programming task, developers were asked to utilise GitHub Copilot, with a treatment group completing the task 55.8% faster than a control group [11]. Despite this, a randomised study querying open-source software developers, working with mature, known codebases in 2025 found completion time increased by 19%, contradicting expected increases in productivity, which were higher [12]. These results indicate that AI excels at greenfield programming tasks, whereas additional reasoning and context is required to parse and measurably improve a highly optimised code base, which is an expected outcome. Indeed, any safety critical improvement would be mired by additional QA confirmation delays, review timeframes, timing analysis and failure mode analysis. These studies indicate why this experiment must measure human active development time, waiting time, review time and rework, as separate considerations, rather than apply a single productivity increase factor, to any later scheduling.

Quality focused studies on LLM enabled development, purport wider potential dangers. Perry et al [13] found participants in a study produced less secure solutions for a given task, whilst exhibiting more confidence in the security and correctness of their implementation. Reasoning ability in models, is not enough to infer the additional context or instruction, where poor prompting or key details are omitted, in a safety critical context. Model knowledge of planning and standards documentation and all pertinent requirements is not enough to confirm usage, orchestration or ‘master-prompting’ must be used to ensure the context window contains the complete extent of information pertinent to any task. NASA’s review of LLM usage in assurance case argument composition indicated available studies do not demonstrate general efficacy in the production of complete safety arguments, in comparison with human produced examples, with time and cost savings being compromised by rework cycles [13, 14].

Agentic Safety Critical Software Development

Agentic approaches represent a step change in the utilisation of LLMs. Web based solutions provide some orchestration capability, but the true extent of what is possible can only be realised once planning, memory, tool usage and role definitions are exploited. The original SWE-bench study [15] found the best agentic model only resolved 1.96% of 2294 issues, in contrast localisation-repair-validation approaches have achieved a 32% completion rate, against the comparable 'SWE-bench Lite' benchmark [16].

Although this provides a decent comparative measure, no consideration is placed towards adherence to outlined programming standards or towards the introduction of defects, increasing risk during operation. The Agentic FEL must be designed such that related system components can be reviewed and evaluated together, to minimise human delays, whilst identifying and removing defects, achievable through SW/SW integration and Unit testing.

It is evident that the tools necessary to permit agentic interaction within a local machine, exist and are freely available. Orchestration frameworks such as MetaGPT [17] and ChatDev [18] organise model instances into roles, for the coordination and parallelisation of tasks, resembling a deployed software engineering team. It is this similarity, which has in part prompted an investigation into the optimal structure of an agentic software team. The technical feasibility of agentic employment for the development of software, against stated requirements, has been demonstrated to a high level, however the holistic performance of these systems in the development of software to a DAL B standard, has not been sufficiently performed.

Orchestration frameworks each operate with unique characteristics and capabilities. Multi-role, singular agent deployment has yielded promising results, evidenced by 'Agentless', where localised, repair and validation loops achieve strong results, without the specialisation or deployment of multiple agents. It is therefore prudent to understand the token cache size, which each agent can utilise, when attempting to solve a problem, according to the corpus of project information such as plans and standards documentation.

Prototyping has confirmed the feasibility of LLM integration with SCADE Suite. Ansys provides the SCADE Python APIs [19], by which process automation can be achieved. Due to the use of additional toolsets, the security boundary of agentic interaction extends, it is therefore prudent to isolate such a model to within the confinement of a virtual machine or sandbox [21, 22, 27]. This allocated aligns with the expected initiation of agentic action, as the fulfilment of a prior action, would trigger agentic execution of the next sequentially blocked item, such a virtual machine would then be created, to permit the execution of the task in a safe environment. NIST defines data governance restrictions which shall be followed in any further deployment of this framework [21]. The NIST SP 800-218A document applies secure development practices to generative AI and model-dependent systems [22]. Relevant controls are included in the Agentic FEL methodology discussion.

Regulatory & Human Factors Position

The proposed EASA's AI Concept Paper [9] defines an approach to software assurance based upon the satisfactory consideration of concepts such as operational and design domains, learning assurance, explainability, risk mitigation and human factors. The UK Military Aviation Authorities (MAA) administrative Regulatory Notice 2025/04 advises similar considerations, including defined operational domains, learning related assessments and architectural mitigation of risk (deterministic bounding/failsafe operation) [23]. The MOD published JSP-936 [24], requires departmental and defence contractor adherence to governance around attributability, through life risk management and dependability assessments. All 3 of these publications support the safe utilisation of AI, recognising the need for additional verification of unique operational characteristics. The provision of such verification evidence requires modification to the traditional FEL, such that these activities can be costed and accounted for during scheduling.

The applicable means of compliance must be agreed and allocated within a defined safety case which safety critical software development already achieves through satisfaction of Def Stan 00-055 [4], and by extension DO-178C [1].

Human gated decision making is not indicative of any generalisable evidence of improved decision quality. Comparatively, the exclusion of required output from an LLMs response, is not evidence of the model's inability to provide such a response. Both scenarios share a common root cause, which is a lack of information and instruction. It is therefore imperative, that any UI design, accounts for a recognition that incorrect prompting can and will occur, information will be excluded, references to standards will be omitted and the full scope of the review may not be correctly specified. As part of agentic orchestration, or AI enablement, project configuration should be performed, such that if any of this information/context is excluded, either a safe approach to locating and using it should be identified, or the request should be blocked.

Parasuraman and Manzey [10] distinguish commission errors, where an incorrect automation recommendation is followed, from omission errors, where contradictory information is not sought. It is the principle of confirmation bias, which may prevent an originator from re-prompting the review of an artefact, if the model initially indicates no errors are present. Such eventualities must be avoided, using context injection, templated artefacts and process specific prompting.

Summary

Existing effort satisfactorily demonstrates the ability of LLM enablement for software engineering, and agentic tool utilisation in incremental development cycles, to a greater level of capability. Despite this, confirmation of AI/Agentic enabled DO-178C/DO-331 assured project execution, with the provision of independence and evidence, has not been demonstrated in the literature, at any significant DAL level. Available benchmarks do not satisfactorily address the scope of the proposed investigation, therefore the identified gap, is a complete deployment and benchmarking of DO-178C compliant, MBSE integrated, LLM enabled processes, which are themselves extensible for use in an agentic orchestration [14-18]. The suggested approach separates the requirement for physical certification credit, from the engineering effort required to achieve the desired outcomes, therefore a reduced software implementation benchmark must be considered.

Methodology Reviews

Scope of Applicability

Two methodologies are selected and reviewed here, against the traditional and state of the art approaches, to the exaction of the full engineering lifecycle (FEL) of a product. According to the scope of this report, RTCA DO-178C [1] shall be selected and applied as the basis for the adaptation of any process, particularly section 11, which defines the software lifecycle data.

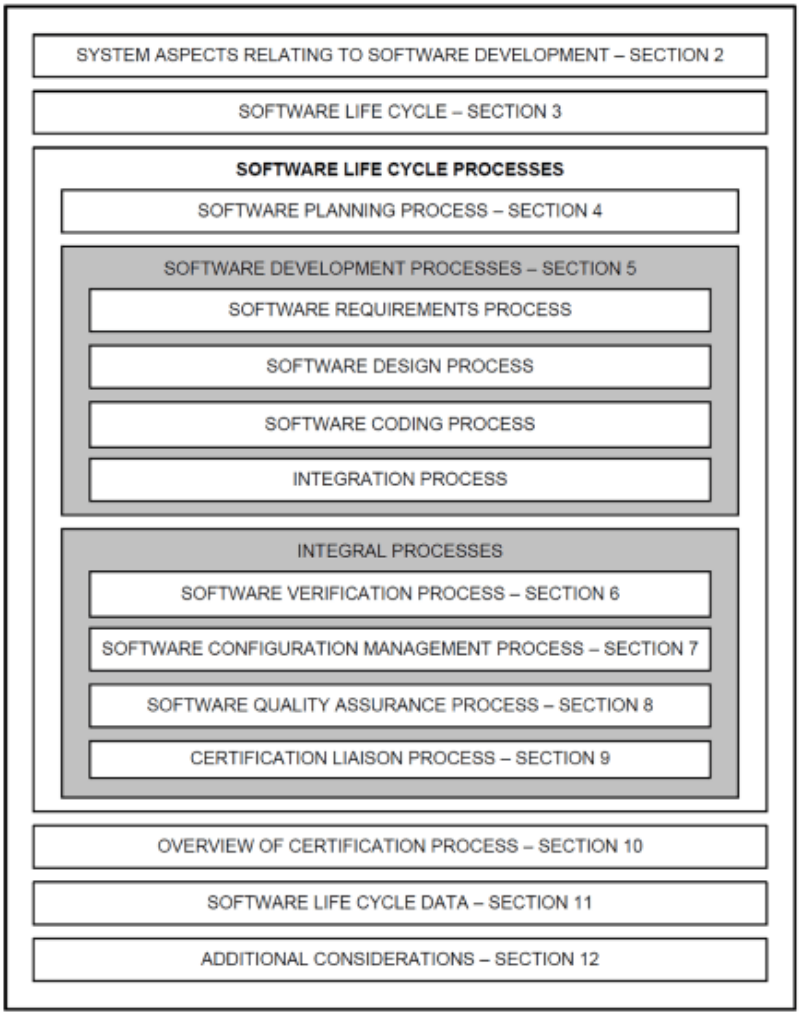

Figure 1 - RTCA DO-178C

To ensure adherence to DO-178C [1] remains viable through the transformation of any traditional process, Section 5 and 6 of Figure 1 are captured within the FEL illustrations below. All artefact associated processes required as per the allocated DAL rating, are preserved.

The methodology reviews compare traditional approaches, with the stated methodologies, for the empirical assessment of performance. Both shall be assessed according to a non-inferiority investigation [28], where compliance to standards, quality, safety, traceability and independence must not be compromised, in any regard.

In this assessment, it is a requirement that Change Control Board (CCB) measures are applied, to retain meaningful modification to requirement baselines, through submission of Software Problem Reports (SPRs) [1, 4].

Methodology Control: Traditional FEL

A control is provided here, in the representation of the traditional FEL, which has been successfully utilised to produce certifiable software, across evolutions of the lifecycles structure, according to the standards selected, to form a software assurance case and to substantiate the means of certification.

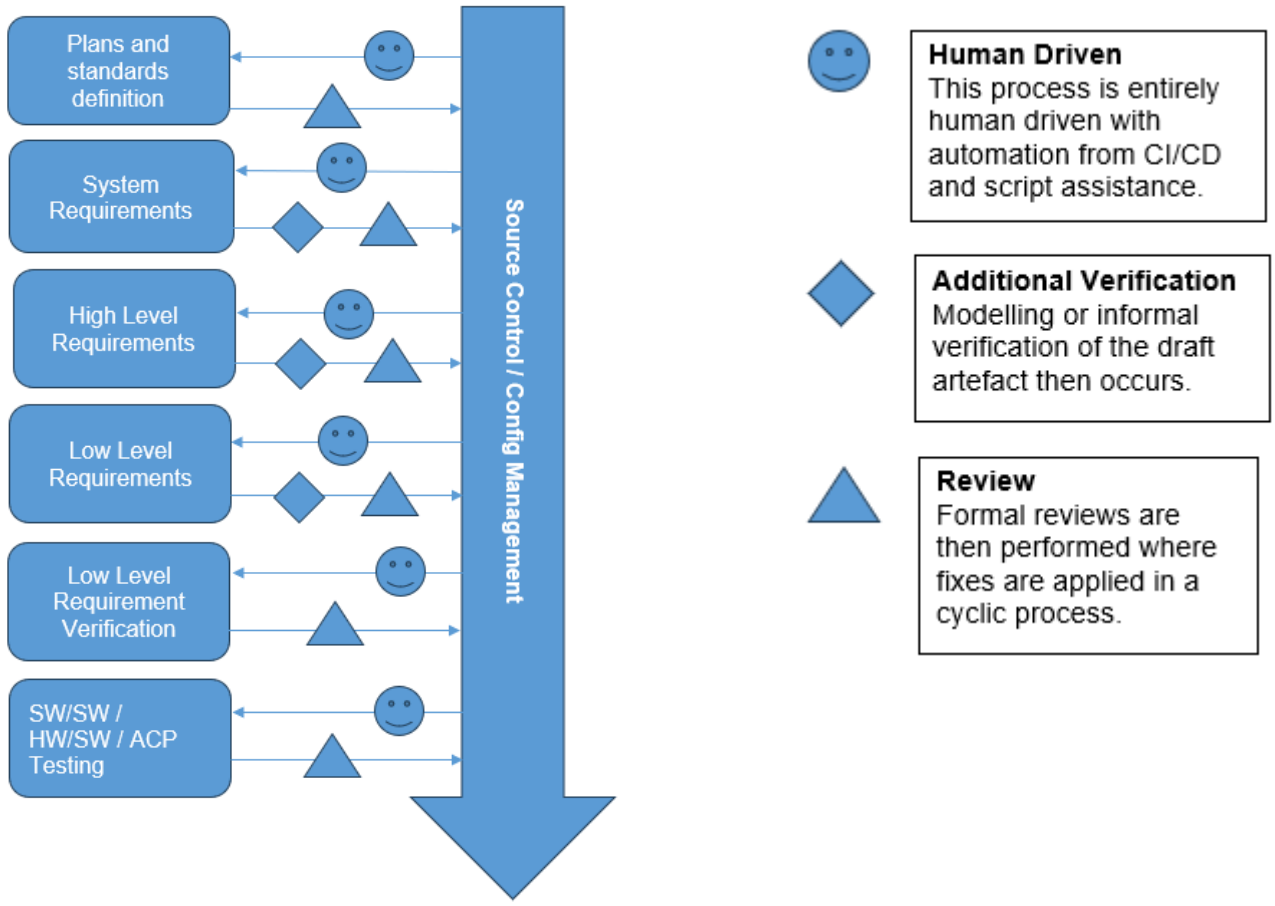

Figure 2 - Traditional FEL Illustration

Plans and standards as documented in 11.1 – 11.8 of Table 1 RTCA DO-178C Lifecycle Components are first created, reviewed and configured. From 11.9 – 11.16, requirements are defined and functional artefacts begin to be created, documented, traced, reviewed, verified and configured. This cycle repeats for each level of requirement decomposition, with baselining occurring at major review stages. Finally, from 11.17 – 11.22, defects are identified, resolved and baselined artefacts are collated, evaluated and configured [1, 3-5].

Failure Modes and Disadvantages

Human omission, or misinterpretation can introduce incorrect requirements, model implementations, or verification implementations, propagating error through requirement baselines if not identified.

Manual traceability can become outdated with each commit as code is modified, the MBSE toolset mitigates this risk in part, but not where manual code is concerned [1 ,6, 7]. Additionally, defects may remain unidentified until late in the lifecycle, or after requirement decomposition , with rework cycles introducing significant delays, costing more as software increases in complexity.

Information categorisation:

Open

Methodology Review 1: LLM Supported Traditional FEL
The FEL of a software product already achieves the delivery of both the software artefact and of all associated assurance material. Therefore, LLM enablement of these existing processes can be implemented with a lesser modification to any material substantiating compliance.

Process Architecture
A high-level definition of an LLM enabled software lifecycle is proposed below.

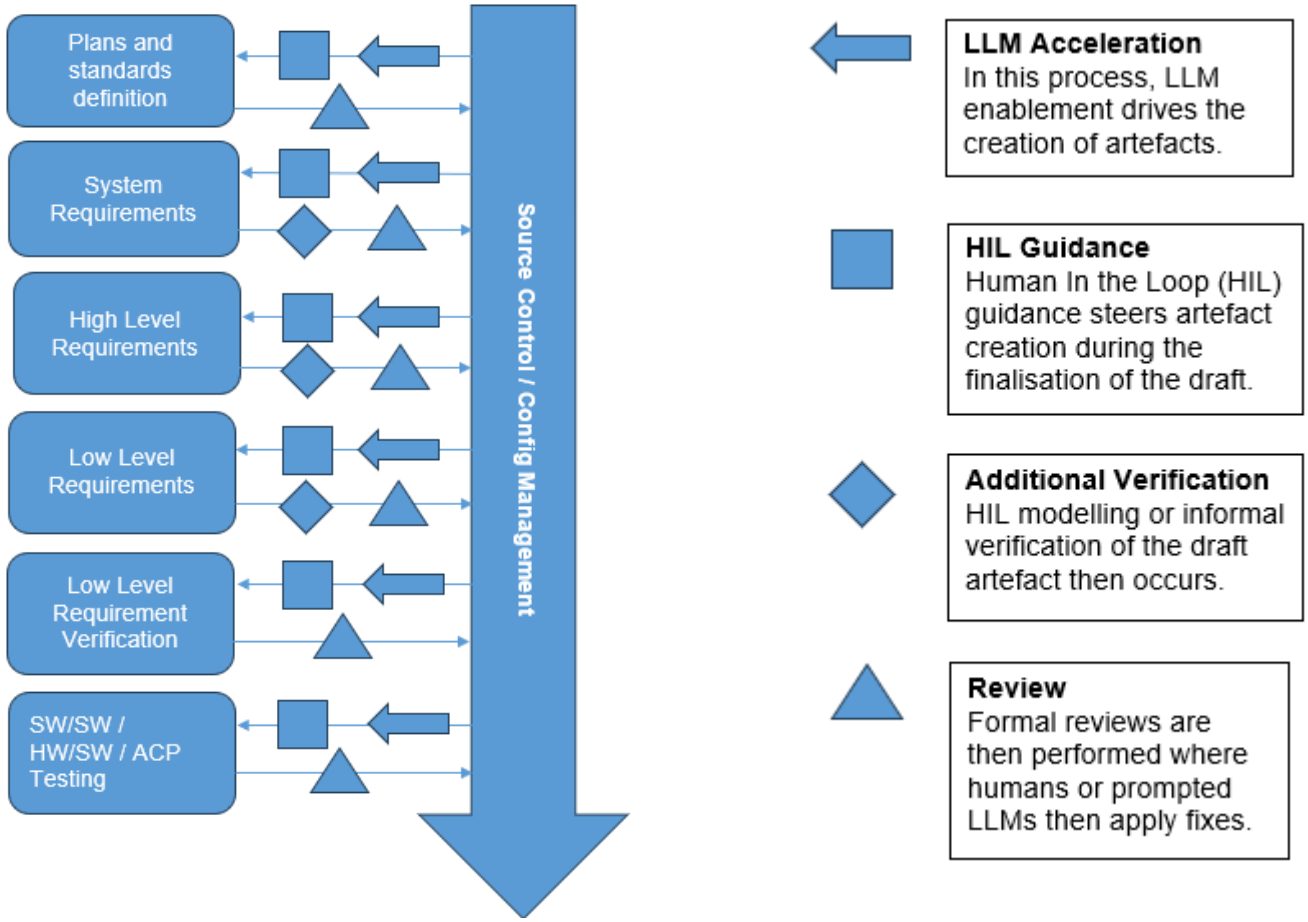

Figure 3 - LLM Enabled FEL Illustration

Assumptions and Prerequisites
This LLM Enabled FEL, adapted from existing examples of DAL A compatible approaches [1, 3] is driven by LLM generation, HIL guidance, verification, modelling and review. The production of traditional FEL associated artefacts requires SQEP, project context, tool knowledge and intimate knowledge of plans and standards. Through token sensitive orchestration, this information can be provided using Retrieval Augmented Generation (RAG), or prompting, thereby accelerating any data dependent process. RAG provides retrieved evidence, against which responses can be grounded, where responses must include accurate reference to any associated requirement, or stipulation within plans and standards documentation [8, 21].

Credibility of Approach
This lifecycle represents the minimum adjustment required to safely integrate this capability into existing processes, the allocation of HIL guidance is necessary to efficiently utilise interruption, and review to avoid wasting time and tokens.

Large Language models, when hosted on premises and load balanced to permit parallel user access, are limited in their capacity, according to hardware limitations such as memory capacity and throughput. According to this limitation, businesses may limit both the size of the model, or the floating point accuracy provided to the model weights and bias node values to improve the cost/value model, therefore frontier model utilisation will be considered as a secondary control when estimating the efficacy of local deployment [29].

Failure Modes and Disadvantages

LLMs are subject to the same prompt/output failures that humans are, through misinterpretation of instruction, defects can be introduced into requirement baselines. The correction of low certainty, through output repair looping, ensures output is validated for correctness, potentially mitigating introduced defects [26]. Incomplete or corrupted data repositories may introduce hallucinations, therefore only reviewed, issued and baselined material should be made available to the RAG capability [8, 27].

The speed with which the production of artefacts is possible may increase the risk of defect propagation, but human gated acceptance, where review is the primary role of the engineer in these circumstances, can be employed to mitigate this early in the lifecycle [10].

IP protection becomes difficult unless the model is severed from the internet, it must retain the entire corpus of knowledge it requires locally, should this be the case, incurring large upfront cost to attain this data, should open-source models not be usable due to licensing restrictions [21, 22].

Lifecycle Modifications

This approach requires additional time spent hosting a model, selecting and defining the project information corpus, and artefact/process specific prompt creation, all upfront. Artefact drafting and rapid prototyping may see acceleration, whilst indirectly increasing review effort, if the complexity of output increases, and if output is not properly evaluated, or tested.

The traditional FEL approaches of attributability can be easily maintained, therefore a standard matrix can be produced. An identifiable hazard here, is the reuse of LLMs as implementors, and reviewers. Models cannot be differentiated as independent actors and are therefore, subject to the defined stipulations around independence proof [1, 2, 5].

Metrics and Acceptance Criteria

This methodology is selected as a stepwise, primarily evaluative tool, for the measurement of LLM acceleration. This benchmark shall be executed by the originator of the manual programming benchmark, so as not to invoke learned experience in the creation of artefacts, neither the LLM nor the originator shall have access to manual benchmark artefacts. Measurable characteristics such as authoring time, search time, tool waiting time, review time, generated tokens, prompting cycles and manual debug time shall be allocated to each artefact.

Quality measures will include establishment of the removal of severity weighted defects, violations of plans and standards, requirement ambiguity, traceability precision and the extent of tool integration. The repeatability or determinism of results shall be established according to a repeat execution of a selection of tasks; it would be infeasible to perform multiple project implementations using LLM enablement alone.

The performance of the RAG capability shall be established separately, to ensure a bottleneck in context awareness is not introduced. The same RAG capability shall be integrated into the agentic orchestration capability, such that no associated tool quality bias is introduced.
Review gates shall be utilised at the delivery of each stage of requirement decomposition, to establish the presence of defects, at those stages.

Summary and Conditions of Rejection
The LLM supported FEL represents a preferable approach to the preservation of attributability, whilst enabling measurement of the effect of LLM support. No presumption of safety or security benefit can be made without analysis of artefacts against stated requirements. According to the utilisation of human review, steering and accountability, existing review and verification measures are expected to mitigate any potential defect introduced by such enablement. Should this claim not be quantifiably substantiated, or should bottlenecks in this process not be identified, the Agentic FEL cannot be justified.

Methodology Review 2: Agentic Team Based FEL
According to the benefits, able to be realised from agentic automation of the development pipeline, the need for human in the loop validation and any requirements to steer/reprompt agentic processes, the existing FEL must be modified.

Process Architecture
A high-level definition of this lifecycle is proposed below.

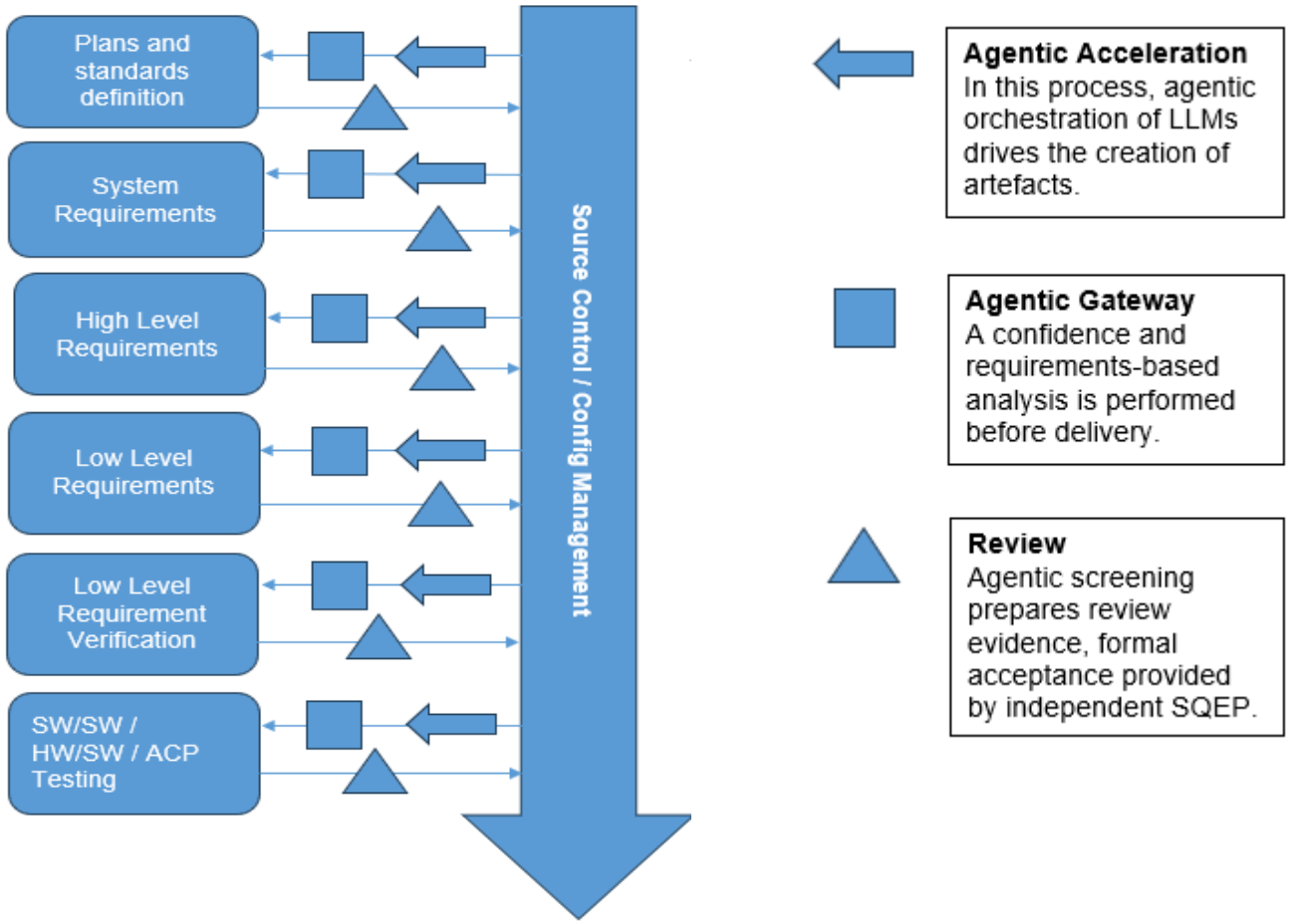

Figure 4 - Agentic FEL Illustration

Assumptions and Prerequisites
Safe parallelisation of tasks is a desired outcome of the traditional FEL, but a variety of factors mean the associated benefits are not exploited in practice. Here, immediately after a requirements baseline is issued, parallel completion of model implementation, informal tests, simulation verification, and traceability analysis can be commenced [16-18]. Gated confirmation stages, where human in the loop steering can be utilised if required, serve as incremental controllers for the acceptance of an artefact. Should the requirements of the task not be met according to an independent verifying agent, or human, incremental development can then be employed to address rework [25, 26].

Credibility of Approach
Under DO-178C, independent verification must be provably utilised in the review of artefacts. The use of separate models does not establish verification independence; any acceptance must be performed by the authorised SQEP who is allocated the responsibility.

The burden of assurance associated with this approach is high, therefore the quality of the plans and standards documentation must be absolute. Agentic output must still be treated as tool output, and automated review must be substantiated by human approval, therefore human attribution must be retained.

Failure Modes and Disadvantages

Incorrect assumptions made at the SRATS level would propagate through each layer, including verification if not caught by a human, therefore system level model integration can be performed, to ensure SRATS are simulated and proven through model-based verification before agentic implementation [17, 18]. The correctness of the source of truth in this mode is paramount.

The initiation of a new cycle can only be triggered after SQEP acceptance, meaning review cycles may still be delayed. Repeatability can be improved in an iterative manner, through model versioning, configurations, explicit prompting and bounded output, traceable back to requirements, however this does not prove model determinism [21].

Successful integration with software lifecycle toolsets is a key requirement, including a need for source control and local sandboxing, should any rollback or recovery capability be permissible.

Lifecycle Modifications

The traditional FEL is interrupted, where work must be sequentially handed off, such that stages of collaborative work can occur. This removes associated hand off delays, however, introduces the risk of accelerated error, or the baselining of an incorrect assumption. Human review gates shall be utilised at the delivery of each stage of requirement decomposition, to establish the presence of defects, at those stages.

Integration of the agentic capability, with all associated toolsets must be achieved, including SCADE itself, Git, Microsoft Office, the operating system and web browsing, as shown by adjacent investigation [19, 21, 22, 25]. External libraries and toolset rich, configurable agentic orchestration frameworks shall be considered for this purpose. Automatic measures for the detection of unauthorised baseline modification, untasked delivery, or non SPR/SCA linked change shall be implemented to prevent illegal change. Where such issues would be dealt with in team meetings, modifications to a corrective list of instructions, with error cases, could be made to update the models knowledge of pitfalls. This would incur maintenance debt over time until a generalised, or specialised (as desired, per the basis of the task) set of instructions is reached.

Metrics and Acceptance Criteria

Specialisation allows for the measurement of model specific task performance, according to role, associated metrics therefore must be designed to establish per task quality. The orchestration model, responsible for marshalling agent deployment and gauging task success, must permit interruption according to lifecycle review gates, where the review agent shall establish this quality. The verifying agent may therefore reject or return an artefact before the human gate is reached, however cannot provide acceptance or approval, lifecycle artefact acceptance remains the responsibility of independent SQEP.

Measurable characteristics such as authoring time, search time, tool waiting time, review time, generated tokens, prompting cycles and manual debug time shall be automatically logged against each artefact. Quality measures will include establishment of the removal of severity weighted defects, violations of plans and standards, requirement ambiguity, traceability precision and the extent of agentic tool integration capability. The repeatability or determinism of results shall be established according to an automatic, repeat execution of a selection of tasks, with the potential for the project to be recreated and compared for the quantification of difference.

Summary and Conditions of Rejection
LLM support presents a more credible, near-term route towards achievement, by preserving established roles, exploiting SQEP and contextual knowledge retained by the human operators.

Agentic support provides vastly improved theoretical capability and concurrency but introduces technical and integration debt, particularly with the increased number of required toolset integrations

The execution of the Agentic FEL offers the greatest theoretical reduction in elapsed time, but also creates the largest assurance, security and human factors change. Adoption of such an approach would require demonstration of an incremental improvement capability, to justify selection of this capability over utilisation of an LLM supported FEL [10, 12-14]. Effective demonstration of non-inferiority and defect identification must be achieved, in addition to provable superiority in comparison to artefacts created using the control, or using an LLM supported FEL [28]. If such conditions cannot be met, or if results are not statistically significant, the safer FEL shall be selected.

Agentic adoption must therefore not be considered, until LLM enabled processes are proven to deterministically produce artefacts of the suitable standard and quality.

Methodology Application and Future Development

Initial Benchmark
The next step in this analysis, involves the production of a controlled benchmark system using manual programming techniques. The chosen benchmark is a Command and Control system for an automatic drone, developed in SCADE Suite and Display, with 4 representative data panels.

An MQTT based communication system shall emulate communication/control of a simulated remote platform. This system and its associated artefacts represent compliance with DAL B objectives [1, 3], with code and documentation stubbed where applicable to permit development in the timeframe allowed.

The chosen example is realistically achievable in a shorter timeframe, because external dependencies are deliberately bounded or emulated, all requirements are defined up front and that stubbing of content will be permitted. The introduced delays here are reflective of experience, with comparable SCADE projects, review timeframes, process design timeframes and other factors.

Traditional estimating techniques and experience were employed to design a schedule for the enaction of the traditional FEL, with literature based schedule hypotheses formed for the additional FELs, included in Table 2 below. Peng et al. [11] identified a 55.8% reduced completion time, but for a non comparative project scope, Cui et al. [20], identified a 26.08% increase across comparable workspaces, but it must be recognised that acceleration of output does not indicate faster acceptance or review, as identified by Perry et al [13]. Should planned benefit be realised without considerable integration/accuracy/performance challenges, the 'Steady Progression' timings apply, 'Validated Use' considers time allocated for rework cycles, given literature reflected partial acceleration of artefact implementation/reviewing [11,12].

Approach	Steady Progression	Validated Use (Incl rework)
Traditional FEL	42 weeks	42 weeks
LLM Enabled FEL	35 weeks	41 weeks
Agentic FEL	30 weeks	40 weeks

Table 2 - C2 Hypothesised Project Schedules

Experiment Design
An independently reviewable package of changes, produced and submitted for review by PR, form the unit of analysis in this experiment. Packages, defined in terms of their comparative functional complexity shall be allocated for completion across the lifecycles. The allocation and order of task implementation shall be randomised (per requirement decomposition stage) to reduce any learning effect with reviewers remaining uninformed as to the origination of the artefact.

The applied system of time logging should distinguish active engineering, context search, prompt construction, deterministic tool execution, model latency, review, rework and waiting. Quality is assessed against the approved requirements and standards, using functional correctness, robustness, model and coding standard compliance, verification adequacy, trace precision and recall, completeness of lifecycle data, and unintended functionality.
The experiment shall utilise a non-inferiority approach, with margins pre-defined for each measurable quality metric before the examination of results, but as a minimum no seeded catastrophic or major defects must remain in the baseline [28].

Table 3 illustrates the planned analyses to be undertaken against each methodology, and the control actions performed to produce a baseline for comparison. Here the human factors requirement was influenced by Parasuraman and Manzeys review [10]. Security assessment requirements shall be defined, according to the associated NIST publications [21, 22].

Analysis	Control Action	Methodology Assessment Action
Purpose Suitability	Establish baseline effort and identify common defects.	Measure effect of AI enabled (LLM) and incremental (agentic) approaches against baseline effort. Agentic approaches shall be tested according to deployed agentic team structure and size.
Timing Benchmark	Complete the engineering lifecycle using standard / stubbed methods, record timings.	Utilise each approach to expedite the lifecycle, record timings. Focus on acceleration for LLM enabled FEL and incremental automation for the agentic FEL.
Achievement	Identify baseline performance in traceability, functionality error, coverage, independent review.	Identify LLM enabled / agentic performance in traceability error, functionality error, coverage, independent review.
Failure Analysis	Perform a failure analysis resulting from seeded, intentional defects.	Identify successful/failed removal of seeded defects, record point of identification and removal.
Human Factors	Perform a baseline human factors assessment to establish good design principles of display components.	Perform the same assessment against LLM enabled and agentic designs. Ensuring the LLMs have knowledge of the assessment standards beforehand.
Security	Identify security requirements and ensure these are followed.	Record and configure evidence of adherence to security requirements, once created.
Decision	Identify which approach resulted in the fewest introduced defects, the highest requirement / implementation / verification quality, in the shortest timeframe.	

Table 3 - Methodology Analysis Matrix

Future Development and Assessment

DO-178C/DO-331 remains the primary acceptance basis [1, 3, 5] for the delivery of any plans and standards documentation, with any other artefact reviewed against the plans and standards themselves. This investigation targets a DAL B level of assurance, therefore limited generalisable findings can be assumed for any other DAL. The benchmark cannot demonstrate compliance or assurance without engagement with authorities; therefore its purpose remains limited to the quantification and analysis of lifecycle and process performance.

It is recognised this investigation is limited by its use of a synthetic and partially stubbed benchmark, additionally the experience of the originator, learning/ordering effects and the quality of the project information corpus may affect results.

According to the potential benefit, both methodologies will be evaluated through the performance of the stated experiment. The production of an empirical method for the establishment of holistic performance, once complete, will provide the necessary evidence to substantiate process transformation in this area. Despite this adoption, engagement with the certification bodies, the

identification of standards to cover usage of AI tooling, and proof of general applicability remain as issues.

Subject to the findings of the stated experiment, the least impactful change, LLM enablement, remains the most defensible position, for short term modification of the FEL via utilisation of existing governance structures, agreed processes and evidence collection methods.
Agentic orchestration constitutes a risky and experimental approach to DAL B+ safety critical process transformation, ill evidenced by the literature [14-18], whose benefit relies upon dependency ridden integration challenges, extensive orchestration [27] and proving security performance [21, 22].

References

1. RTCA (2011a) *DO-178C: Software Considerations in Airborne Systems and Equipment Certification*. Washington, DC: RTCA, Inc. Available at: RTCA software standards

2. RTCA (2011b) *DO-330: Software Tool Qualification Considerations*. Washington, DC: RTCA, Inc. Available at: RTCA DO-330.

3. RTCA (2011c) *DO-331: Model-Based Development and Verification Supplement to DO-178C and DO-278A*. Washington, DC: RTCA, Inc. See: FAA recognition of DO-331.

4. Ministry of Defence (2021) *Defence Standard 00-055 Part 1, Issue 5: Requirements for Safety of Programmable Elements (PE) in Defence Systems—Part 1: Requirements and Guidance*. Glasgow: UK Defence Standardization.

5. Federal Aviation Administration (2017) *AC 20-115D: Airborne Software Development Assurance Using EUROCAE ED-12() and RTCA DO-178()*. Washington, DC: FAA. Available at: FAA AC 20-115D

6. Ansys (2025a) 'Efficient Development of Safe Avionics Software with DO-178C Objectives Using SCADE Suite', 9 April. Available at: Ansys Knowledge

7. Ansys (2025b) 'Efficient Verification of Safe Avionics Software with DO-178C Objectives Using SCADE Suite', 19 May. Available at: Ansys Knowledge

8. Lewis, P. et al. (2020) 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks', Advances in Neural Information Processing Systems, 33, pp. 9459–9474. Available at: arXiv:2005.11401.

9. European Union Aviation Safety Agency (2026) EASA Artificial Intelligence Concept Paper—Proposed Issue 3. Cologne: EASA. Available at: EASA Concept Paper.

10. Parasuraman, R. and Manzey, D.H. (2010) 'Complacency and Bias in Human Use of Automation: An Attentional Integration', Human Factors, 52(3), pp. 381–410. doi:10.1177/0018720810376055.

11. Peng, S., Kalliamvakou, E., Cihon, P. and Demirer, M. (2023) 'The Impact of AI on Developer Productivity: Evidence from GitHub Copilot', arXiv:2302.06590. Available at: arXiv.

12. Becker, J., Rush, N., Barnes, E. and Rein, D. (2025) 'Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity', arXiv:2507.09089. Available at: METR study.

13. Perry, N., Srivastava, M., Kumar, D. and Boneh, D. (2023) 'Do Users Write More Insecure Code with AI Assistants?', Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. doi:10.1145/3576915.3623157.

14. Graydon, M.S. and Lehman, S.M. (2025) Examining Proposed Uses of LLMs to Produce or Assess Assurance Arguments. NASA/TM-20250001849. Hampton, VA: NASA. Available at: NASA Technical Reports Server.

15. Jimenez, C.E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O. and Narasimhan, K. (2024) 'SWE-bench: Can Language Models Resolve Real-World GitHub Issues?', Proceedings of the Twelfth International Conference on Learning Representations. Available at: arXiv:2310.06770.NIST's Generative AI Profile 2024

16. Xia, C.S., Deng, Y., Dunn, S. and Zhang, L. (2025) 'Agentless: Demystifying LLM-based Software Engineering Agents', *Proceedings of the 33rd ACM SIGSOFT International Symposium on the Foundations of Software Engineering*. Available at: arXiv:2407.01489.

17. Hong, S. et al. (2024) 'MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework', *Proceedings of the Twelfth International Conference on Learning Representations*. Available at: arXiv:2308.00352.

18. Qian, C. et al. (2024) 'ChatDev: Communicative Agents for Software Development', *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 1, pp. 15174–15186. Available at: arXiv:2307.07924.

19. Ansys (2024) 'Using the SCADE Python APIs from Your Favorite IDE', 3 January. Available at: Ansys Knowledge.

20. Cui, K.Z., Demirer, M., Jaffe, S., Musolff, L., Peng, S. and Salz, T. (2026) 'The Effects of Generative AI on High-Skilled Work: Evidence from Three Field Experiments with Software Developers', *Management Science*, advance online publication. doi:10.1287/mnsc.2025.00535.

21. National Institute of Standards and Technology (2024a) *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*. NIST AI 600-1. Gaithersburg, MD: NIST. Available at: NIST AI 600-1.

22. National Institute of Standards and Technology (2024b) *Secure Software Development Practices for Generative AI and Dual-Use Foundation Models: An SSDF Community Profile*. NIST SP 800-218A. Gaithersburg, MD: NIST. doi:10.6028/NIST.SP.800-218A.

23. Military Aviation Authority (2025) *MAA/RN/2025/04: Use of Artificial Intelligence within Systems and Equipment Subject to MRP Design Assurance Processes*. Bristol: Ministry of Defence. Available at: MAA Regulatory Notices.

24. Ministry of Defence (2024) *JSP 936: Dependable Artificial Intelligence in Defence—Part 1: Directive*. London: Ministry of Defence. Available at: JSP 936.

25. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K.R. and Cao, Y. (2023) 'ReAct: Synergizing Reasoning and Acting in Language Models', *Proceedings of the Eleventh International Conference on Learning Representations*. Available at: arXiv:2210.03629.

26. Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K.R. and Yao, S. (2023) 'Reflexion: Language Agents with Verbal Reinforcement Learning', *Advances in Neural Information Processing Systems*, 36. Available at: arXiv:2303.11366.

27. Zou, W., Geng, R., Wang, B. and Jia, J. (2025) 'PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models', *Proceedings of the 34th USENIX Security Symposium*, pp. 3827–3844. Available at: USENIX Security 2025.

28. Lakens, D. (2017) 'Equivalence Tests: A Practical Primer for t Tests, Correlations, and Meta-Analyses', *Social Psychological and Personality Science*, 8(4), pp. 355–362. doi:10.1177/1948550617697177.

29. Dettmers, T., Lewis, M., Belkada, Y. and Zettlemoyer, L. (2022) 'LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale', *Advances in Neural Information Processing Systems*, 35. Available at: arXiv:2208.07339.